Design Report

Panda Scooter Project

Panda Scooter Team

Contents

1 Introduction

This report explains the design of Panda Scooter, a shared scooter platform that supports riders, dispatchers, and administrators in a unified workflow.

The system was designed to address the operational complexity of shared scooter management. A complete ride service must coordinate user authentication, scooter discovery, unlocking and locking, billing, fault reporting, vehicle dispatch, and administrative maintenance. A single-role application is not sufficient for these tasks, so Panda Scooter is implemented as a multi-app system with three frontends, one backend service, and one shared database.

The system scope of this report is summarized below.

Component	**Scope description**
Rider mobile app	Scooter discovery, unlocking, riding, billing, fault reporting, wallet access, and personal record viewing.
Dispatcher mobile app	Vehicle lookup, scooter unlock/lock support, operational record viewing, and scooter information lookup.
Administrator web app	Fleet monitoring, zone management, parking point management, no-parking zone management, pricing, packages, and dispatcher management.
Backend service	Unified REST API, role-based authentication, business rules, MQTT integration, and data processing.
Database	Persistent storage for users, scooters, orders, wallets, subscriptions, zones, parking resources, and reports.

Table 1: System scope of the Panda Scooter project.

The remainder of this report is organized as follows. Section 2 gives an overall overview of the project. Section 3 describes the system architecture. Sections 4 to 6 present module, database, and interface design. Section 7 summarizes the non-functional design considerations, and Section 8 concludes the report.

2 Overall Overview

2.1 Project Positioning

Panda Scooter is a shared scooter management platform for a three-role operational workflow. It supports rider-facing ride activities, dispatcher-side scooter handling, and administrator-side fleet management in one integrated system. The design covers an end-to-end business flow.

2.2 Users and Runtime Environments

The system is used by three types of users in different runtime environments:

Role	Client	Runtime	Main usage
Rider	Mobile app	uni-app	Scooter discovery, unlock and lock, wallet, bills, fault reporting, and ride records.
Dispatcher	Mobile app	uni-app	Vehicle lookup, dispatch operations, scooter information, and dispatch history.
Administrator	Web app	Browser	Dashboard monitoring, zones, parking points, packages, pricing, and dispatcher management.

Table 2: User roles and runtime environments.

2.3 Technology Profile

The overall technology profile is summarized below:

Layer	Technology set
Mobile frontend	uni-app
Web frontend	Vue 3, AMap JSAPI
Backend	Spring Boot 3.2.5, Java 17, MyBatis, MQTT, Redis, MinIO
Database	MySQL 8.0
Documentation	SpringDoc/OpenAPI

Table 3: Technical profile of the Panda Scooter system.

2.4 Deployment Model and Constraints

The deployment model separates the system into three client deployments, one backend service, one database, and several external integration services. The rider and dispatcher clients run as WeChat mini-programs, the administrator client runs in a browser, and all three communicate with the same Spring Boot backend through HTTP. The backend interacts with MySQL, MQTT, AMap, and MinIO according to the needs of each business flow.

Constraint type	Main constraints
Functional constraints	Strict role isolation, map-based scooter and zone queries, and MQTT-based scooter command handling.
Non-functional constraints	A maintainable data model, independent frontend evolution, and a scope that remains manageable within the current project.

Table 4: Main design constraints of the project.

These constraints drive the architecture and module organization described in the next section.

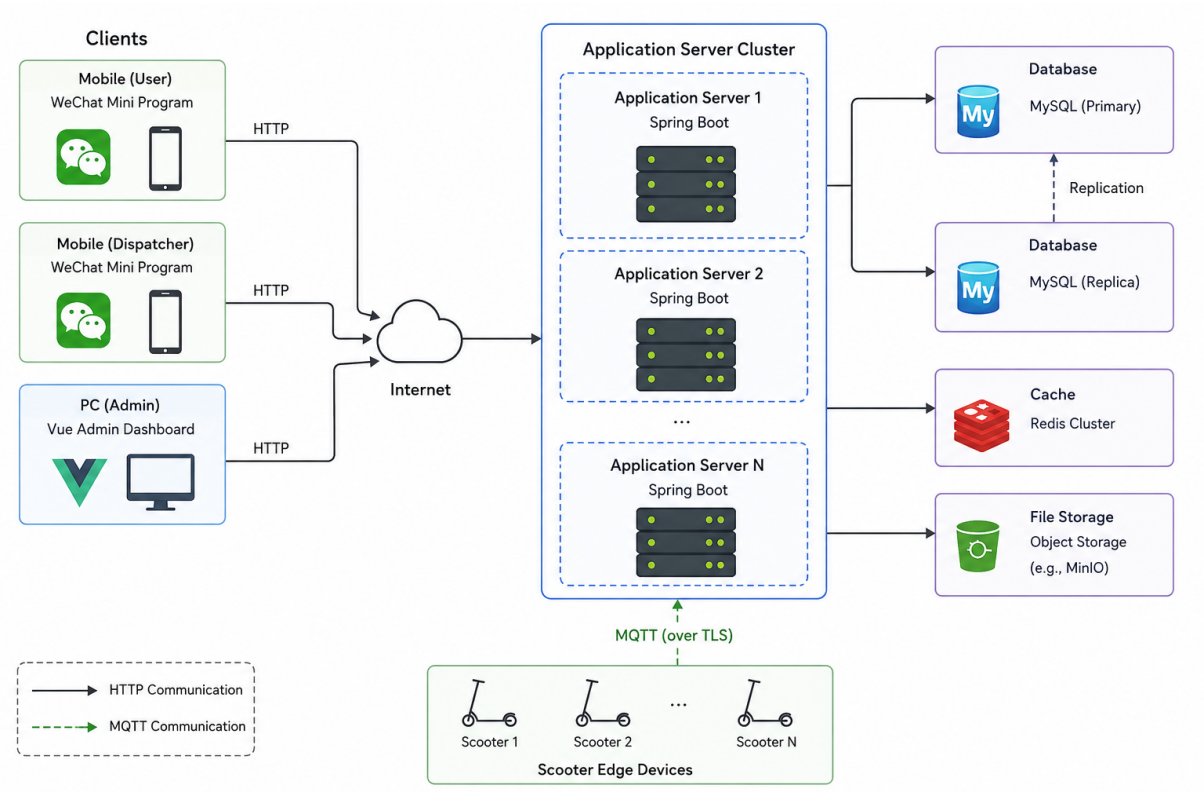

Figure 1: System deployment layout showing the rider mini-program, dispatcher mini-program, browser-based admin client, shared backend service, MySQL database, MQTT communication, AMap services, and MinIO storage.

3 System Architecture Design

3.1 Architecture Overview

The system follows a Browser-Server architecture. The rider and dispatcher applications are deployed as WeChat mini-programs, while the administrator interface runs in a browser-based web environment. This layout matches the project workflow: mobile users

perform operational tasks on the move, and administrators manage fleet resources in a desktop-style control panel.

All client requests are routed to a shared Spring Boot backend, which acts as the central business hub of the system. The backend handles authentication, business processing, persistence access, and external service integration, so the frontends can remain lightweight and role-focused.

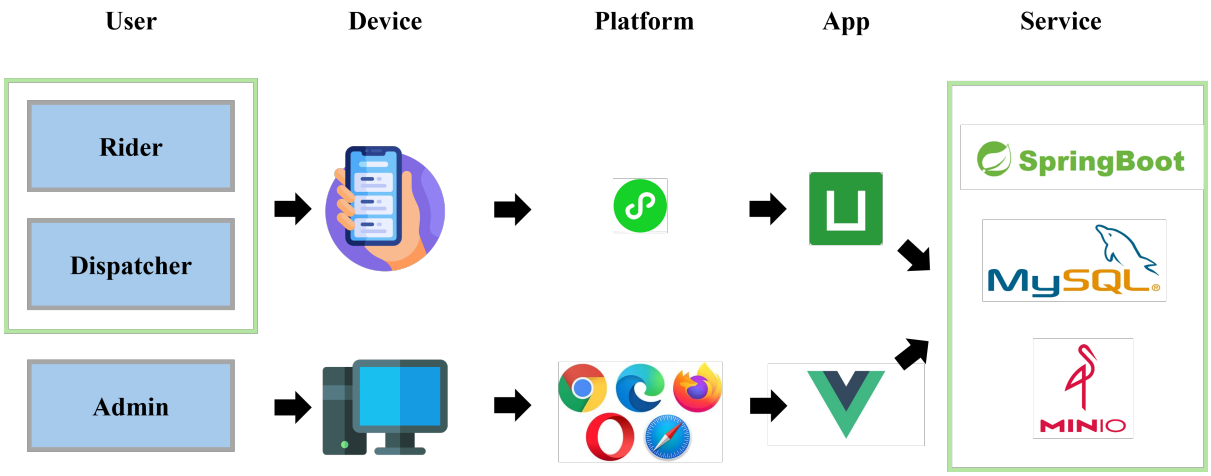

Figure 2: System architecture overview showing the rider mini-program, dispatcher mini-program, browser-based admin client, shared backend service, MySQL database, MQTT communication, AMap services, and MinIO storage.

3.2 Client Layer Design

The client layer consists of three role-specific applications, each designed for a different operational context.

Client	Responsibility
Rider mini-program	Scooter discovery, unlocking, riding, billing, and fault reporting.
Dispatcher mini-program	Scooter lookup, dispatch actions, scooter status checks, and dispatch history review.
Admin browser client	Dashboard monitoring and the management of zones, parking points, scooters, packages, pricing rules, and dispatcher accounts.

Table 5: Responsibilities of the client layer.

The separation of clients reduces interface clutter and keeps each front end aligned with the responsibilities of its target users. It also allows the mobile and browser clients to evolve independently without affecting the shared backend contract.

3.3 Backend Service Design

The backend is implemented as a Spring Boot service and is organized into three Maven modules:

Module	Role in the backend
`panda-common`	Shared constants, JWT utilities, exception classes, base context, and response wrappers.
`panda-pojo`	Entities, DTOs, and VOs that describe the system data contracts.
`panda-server`	Executable application with controllers, services, interceptors, configuration, and MQTT logic.

Table 6: Backend module responsibilities.

This structure separates reusable support code, data definitions, and runtime business logic. Controllers receive requests from the clients, services apply the business rules, and mappers transfer data between the application and the database. The backend therefore acts as the single coordination point for the entire platform.

3.4 Data and Integration Layer Design

The data and integration layer provides the persistent and external services required by the platform.

Service	Type	Role in the system
MySQL	Database	Stores users, scooters, orders, wallet records, subscriptions, zones, parking points, and fault reports.
MQTT	Messaging layer	Delivers scooter commands and supports telemetry exchange.
AMap	Map service	Supports map display and location-based scooter or zone queries.
MinIO	File storage	Stores uploaded files, especially fault-report images.

Table 7: Data and integration services.

These services support the core business flow without forcing the frontend clients to manage low-level persistence or device communication directly. They also make the architecture easier to extend because each integration point is isolated in a dedicated service.

3.5 Security and Access Isolation

Security is enforced through role-based route separation and JWT-based interception. In `WebMvcConfiguration`, requests to `/user/**` are processed by the rider interceptor,

requests to `/dispatcher/**` are processed by the dispatcher interceptor, and requests to `/admin/**` are processed by the admin interceptor.

Public routes such as login and verification are excluded from token checks so that authentication can complete before authorization is applied. This design keeps the security boundary clear while still allowing the user-facing login flow to remain accessible.

The architecture therefore combines role isolation, centralised backend control, and separated client responsibilities, which fits the shared-scooter workflow represented in the architecture diagram.

4 Module Design

The source code reveals a clear functional separation between rider, dispatcher, and admin modules. In addition, ride processing, billing, and MQTT communication are implemented as reusable backend services. Table 8 gives a concise module overview before the detailed description of each module.

Module	**Primary focus**	**Typical outputs**
Rider module	Ride lifecycle and personal service functions	Scooter discovery, unlock/lock, wallet, bills, faults, subscriptions, and ride records.
Dispatcher module	Operational scooter handling	Scooter lookup, dispatch actions, scooter information, and dispatch history.
Admin module	Platform maintenance and monitoring	Dashboard data, fleet management, zones, parking points, packages, pricing, and dispatcher administration.
Shared backend services	Cross-module business support	Ride processing, billing, and MQTT-based device communication.

Table 8: Overview of the major modules in the system.

4.1 Rider Module

The rider module is the user-facing entry point for the ride workflow. It focuses on the complete rider journey from account access to ride settlement.

Function area	Main pages	Key behavior
Account management	`login`, `resetPassword`, `profile`, `account`	Registration, login, logout, deletion, password reset, and verification.
Ride discovery	`index`, `parking`, `unlock`	Map-based scooter discovery, parking point browsing, and unlock initiation.
Ride execution	`riding`, `unlock`, `lock`	Active ride display, ride termination, and order closure.
Finance and history	`wallet`, `bill`, `history`	Wallet balance, bill records, and ride history.
Fault and package support	`faults`, `reportFault`	Fault browsing, fault submission, and image upload.

Table 9: Rider module structure.

The rider module is centered on the operational sequence of finding a scooter, starting a ride, ending the ride, and reviewing the settlement result. It also includes supporting functions such as fault reporting and subscription browsing.

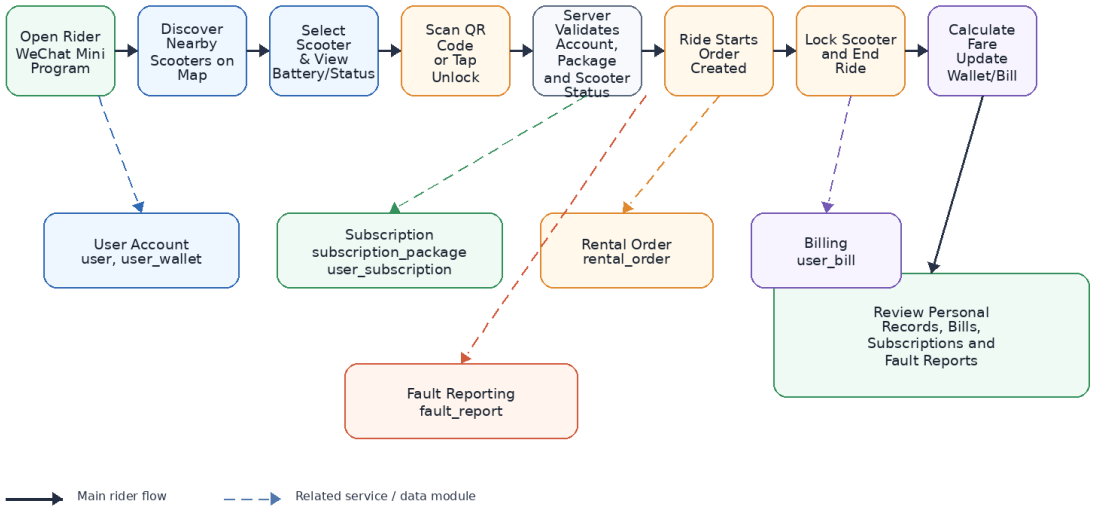

Figure 3: Rider module flow from scooter discovery to unlock, riding, lock, billing, and personal record review.

4.2 Dispatcher Module

The dispatcher module supports the operational staff who manage scooters in the field. Compared with the rider module, it is narrower in scope but more focused on vehicle handling and status checking.

Function area	Main pages	Key behavior
Account management	`login`, `resetPassword`, `profile`, `account`	Registration, login, logout, deletion, verification, and password reset.
Operational lookup	`index`, `vehicleLookup`, `scooterInfo`	Map inspection, scooter lookup by code, and scooter information viewing.
Field operations	`unlock`, `lock`	Operational scooter unlock and lock actions.
Record tracing	`history`	Dispatch history and traceability of operations.

Table 10: Dispatcher module structure.

The dispatcher module is designed to make scooter handling quick and traceable. Its interface is smaller than the rider module, but each page is tied directly to a field operation.

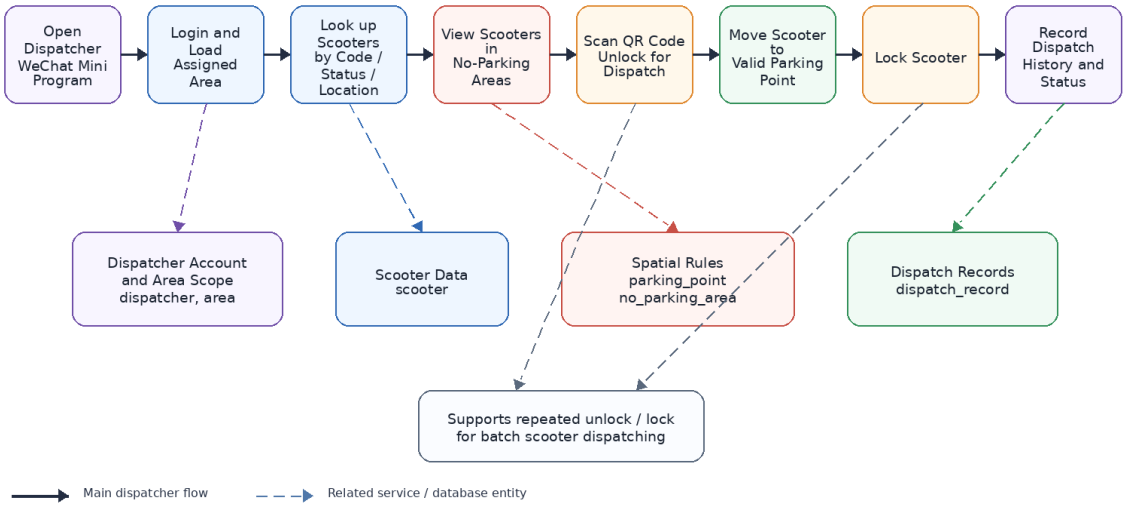

Figure 4: Dispatcher module flow from scooter lookup to unlock, lock, and dispatch record tracking.

4.3 Admin Module

The admin module is the platform maintenance center. It organizes the management functions around the resources that need to be configured and monitored.

Resource area	Main views	Key behavior
Monitoring	`DashboardView`	Dashboard overview and live data inspection.
Fleet management	`VehicleManagementView`	Scooter list management.
Spatial management	`ZonesView`, `ZoneEditorView`, `ZoneDetailView`	Zone creation, editing, detail viewing, and deletion.
Parking control	`ParkingPointsView`, `ParkingPointEditorView`, `NoParkingZonesView`, `NoParkingZoneEditorView`	Parking point and no-parking area management.
Commercial settings	`PricingView`, `PackagesView`	Pricing rule and subscription package management.
Staff administration	`DispatchersView`, `LoginView`	Dispatcher administration and admin login functions.

Table 11: Admin module structure.

The admin module is organized by resource type rather than by user journey. This makes the management interface more systematic and easier to extend when new operational resources are introduced.

4.4 Shared Backend Services

The shared backend services are not tied to a single frontend. They support the business rules that are reused by all three modules.

Service	Main responsibility	Supported modules
Ride service	Scooter lookup, unlock, lock, map data retrieval, unpaid-order payment, and ride history	Rider and dispatcher modules
Billing service	Wallet deduction, bill generation, and package-related settlement	Rider module and admin reporting
MQTT service	Publisher, listener, retry, and online-status support for scooter communication	Rider and dispatcher operational commands

Table 12: Shared backend services used across the modules.

The ride service contains the most important business rules of the platform, such as one active ride per user, no unlock with unpaid orders, and ride settlement after locking. The MQTT layer enables asynchronous command delivery and telemetry exchange.

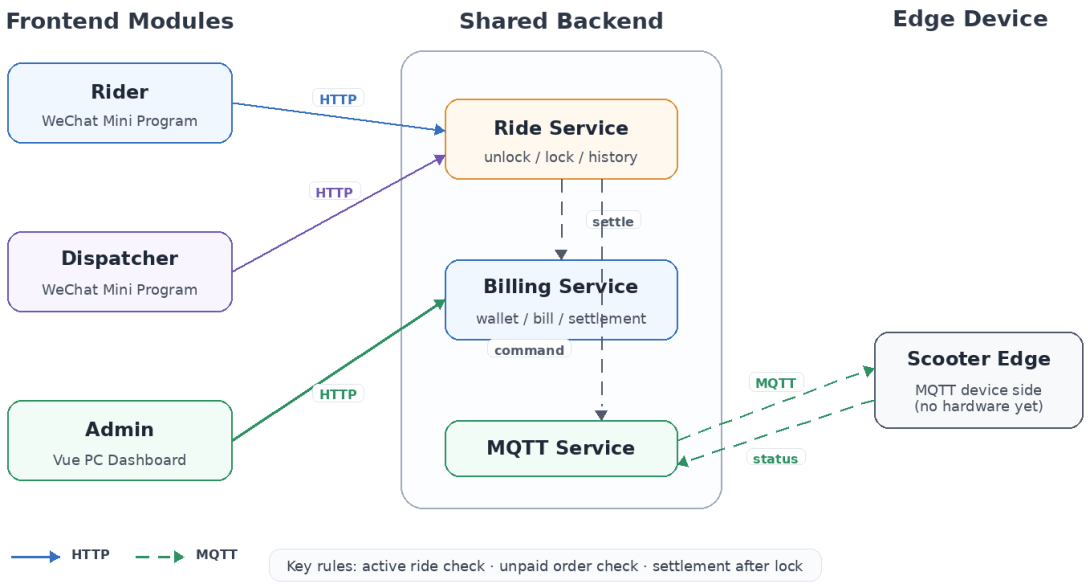

Figure 5: Relationship diagram showing the rider, dispatcher, admin, ride service, billing service, and MQTT service connections.

5 Database Design

The database schema is defined in `docs/design report/bike_system.sql`. The design is centered on the main business entities of the platform. The table structure is easy to map to the source code because most entities have direct Java counterparts in `panda-pojo`.

5.1 Main tables

Table	Key fields	Purpose
`admin`	`id`, `username`, `password`, `email`, `create_time`	Administrator account information.
`user`	`id`, `username`, `password`, `email`, `create_time`	Rider account information.
`dispatcher`	`id`, `name`, `password`, `email`, `area_id`, `status`	Dispatcher identity and assigned area.

Table	Key fields	Purpose
scooter	id, code, battery, latitude, longitude, ride_status, fault_status	Scooter status and location.
rental_order	user_id, scooter_id, start_time, end_time, total_time, order_status, pay_status, amount	Ride lifecycle and settlement.
package_order	user_id, package_id, price, order_status, pay_status	Subscription package purchase orders.
dispatch_record	dispatcher_id, scooter_id, status, end_time	Dispatcher operations history.
user_wallet	user_id, balance, update_time	User balance storage.
user_bill	user_id, type, amount, balance_after, order_id, remark	Wallet transaction ledger.
subscription_package	title, description, price, type	Package catalog.
user_subscription	user_id, package_id, start_time, end_time, status	Package ownership and validity.
parking_point	name, latitude, longitude, status	Legal parking locations.
no_parking_area	name, polygon, status	Forbidden parking polygons.
area	name, polygon, create_time	Operational area definition.
pricing_rule	price_per_min, base_price, billing_interval	Ride pricing configuration.

Table	Key fields	Purpose
`fault_report`	`user_id`, `scooter_id`, `description`, `image_url`, `status`	Fault reporting.

5.2 Relationship and indexing design

The schema uses indexes on fields that are frequently queried in business flows, such as `user_id`, `scooter_id`, `dispatcher_id`, and `order_id`. This supports ride lookup, wallet reconciliation, dispatch record queries, and audit-like operations.

The `scooter` table stores both identity and runtime state, which makes map display and operational control straightforward. The `rental_order` and `user_bill` tables form the financial core of the system, while `user_subscription` records time-bound package ownership.

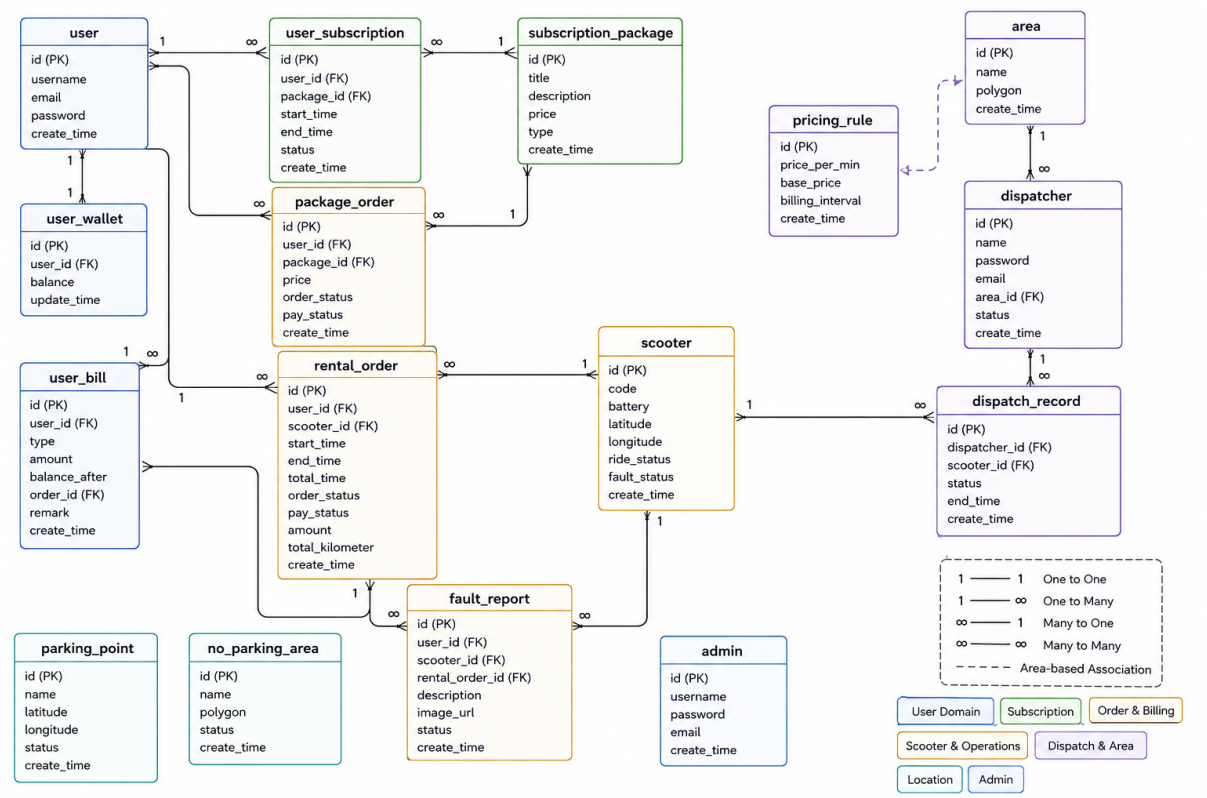

Figure 6: ER diagram showing the relations among riders, dispatchers, scooters, orders, bills, subscriptions, zones, and parking resources.

6 Interface Design

The backend uses REST-style interfaces and a unified response wrapper, `Result<T>`, to keep response handling consistent across the frontends. The design also uses SpringDoc,

which supports interface documentation and interactive API inspection.

6.1 API inventory compiled from the documentation

The following API list is compiled from the generated documentation files in `docs/design report`. Every documented endpoint is included in the report and grouped by role and function so that the interface structure is easy to trace. Some account-related routes are intentionally shared across rider and dispatcher clients because both frontends reuse the same login and verification flow.

Role	Endpoint count	Design focus
Rider	19	Account lifecycle, ride flow, wallet, bills, faults, and subscriptions.
Dispatcher	12	Shared account flow, dispatch operations, map queries, and scooter status lookup.
Administrator	28	Monitoring, pricing, package, zone, no-parking, dispatcher, parking point, and fleet management.

Table 14: Coverage summary of the documented APIs.

6.1.1 Rider API inventory

Function group	Documented endpoints	Purpose
Account lifecycle	`POST /user/signin`, `POST /user/login`, `POST /user/logout`, `DELETE /user/delete`, `GET /user/verification`, `POST /user/password`	Rider registration, login, logout, deletion, verification, and password reset.
Wallet and bills	`POST /user/bill`, `GET /user/bills`, `GET /user/wallet`	Wallet charge and consumption records, bill history, and wallet balance.

Function group	Documented endpoints	Purpose
Ride and map	`GET /ride-history`, `POST /unpaid-order`, `GET /user/info`, `GET /map`, `GET /scooter`, `POST /scooter/unlock`, `POST /scooter/lock`	Ride lookup, scooter discovery, map query, unlock/lock, unpaid-order payment, and ride information.
Fault and subscription	`GET /faults`, `POST /fault`, `GET /subscription`	Fault report browsing, fault submission, and subscription package browsing.

- The rider API set supports the full lifecycle from account creation to trip completion.
- The wallet and bill endpoints form the financial trail used by the ride settlement flow.
- The fault and subscription routes extend the rider experience beyond the core riding scenario.

6.1.2 Dispatcher API inventory

Function group	Documented endpoints	Purpose
Account lifecycle	`POST /user/signin`, `POST /user/login`, `POST /user/logout`, `DELETE /user/delete`, `GET /user/verification`, `POST /user/password`	Dispatcher registration, login, logout, deletion, verification, and password reset.
Operational information	`GET /user/info`, `GET /user/dispatch-history`	Dispatcher profile information and dispatch history.

Function group	Documented endpoints	Purpose
Map and scooter operations	`GET /map`, `POST /scooter/unlock`, `POST /scooter/lock`, `GET /scooter/info`	Map query, scooter unlock/lock, and scooter information lookup.

- Dispatcher endpoints reuse the same authentication model as riders, which keeps the account workflow consistent across mobile clients.
- Operational APIs are concentrated around map inspection and scooter command execution.

6.1.3 Admin API inventory

Function group	Documented endpoints	Purpose
Authentication	`POST /admin/login`, `POST /admin/logout`	Admin login and logout.
Monitoring	`GET /admin/overview`, `GET /admin/live-data`	Dashboard metrics and real-time data.
Pricing	`GET /admin/pricing-rules`, `PUT /admin/pricing-rules`	Pricing rule retrieval and updates.
Packages	`GET /admin/packages`, `POST /admin/packages`, `PUT /admin/packages/{id}`, `DELETE /admin/packages/{id}`	Subscription package management.
Zones	`GET /admin/zones`, `POST /admin/zones`, `GET /admin/zones/{id}`, `PUT /admin/zones/{id}`, `DELETE /admin/zones/{id}`	Operational zone management.
No-parking zones	`GET /admin/no-parking-zones`, `POST /admin/no-parking-zones`, `PUT /admin/no-parking-zones/{id}`, `DELETE /admin/no-parking-zones/{id}`	No-parking polygon management.
Dispatchers	`GET /admin/dispatchers`, `POST /admin/dispatchers`, `PUT /admin/dispatchers/{id}`, `DELETE /admin/dispatchers/{id}`	Dispatcher account management.

Function group	Documented endpoints		Purpose
Parking points	`GET /admin/parking-points`, `/admin/parking-points`, `/admin/parking-points/{id}`, `/admin/parking-points/{id}`	`POST` `PUT` `DELETE`	Parking point management.
Scooters	`GET /admin/scooters`		Fleet list query.

- The admin API set is the largest group because it combines monitoring and maintenance functions.
- Management endpoints are arranged by resource type so the dashboard can mirror the operational structure of the system.

6.2 Authorization

The platform uses JWT-based authorization to separate the three roles and protect their private routes. After successful login, the backend issues a token that is carried by the client in subsequent requests. The token is stored in the client and attached to protected requests so that the backend can identify the current role and enforce access control.

The token payload includes the core identity claims needed by the backend, such as the user identifier, username or name, role information, and token timing information. In practice, the payload is encoded into a compact JWT structure, which is then transmitted as a bearer token. The backend decodes the token, verifies its signature, and extracts the claims before the request is allowed to reach the business layer.

JWT claim	Meaning
`id`	Unique identifier of the authenticated account.
`username` or `name`	Display identity of the current account.
`role`	Role label used for access control, such as rider, dispatcher, or administrator.
`iat`	Issued-at timestamp used by the token library.
`exp`	Expiration timestamp used to limit token lifetime.

Table 18: Typical information carried in the JWT payload.

The protected namespaces are `/user/**`, `/dispatcher/**`, and `/admin/**`. Public routes such as login and verification remain open so that the token can be obtained before authorization is applied.

6.3 Response format

The backend returns a unified `Result<T>` wrapper for all interfaces. This design keeps the client-side handling logic simple because the frontends only need to check one response structure regardless of the business function being called.

Response field	Meaning
`code`	Status code used by the frontend to judge whether the request succeeded.
`msg`	Human-readable message returned by the backend.
`data`	Business payload returned to the client.

Table 19: Common response wrapper structure.

Response type	Typical role in the system
Success response	Carries the correct code '0' and the requested business data.
Failure response	Carries the error code '1' and message used to explain why the request failed.

Table 20: Response behavior in the unified wrapper.

7 Non-functional Design

The non-functional design covers security, performance, maintainability, and reliability.

7.1 Security design

Security is implemented through role-based JWT interceptors and route separation. Each role has its own protected namespace, which reduces the risk of cross-role access. CORS configuration is also defined globally so that the frontend applications can communicate with the backend during development and deployment.

Password handling is implemented through server-side processing before storage. Validation is used to reduce malformed requests, and the global exception handling strategy keeps error responses predictable.

7.2 Performance design

The design uses index-supported relational tables and bounded map queries. The scooter map data is filtered by the current map center and scale, which avoids returning an unnecessary full-data set. The admin data overview and live data endpoints are also query-driven so that the dashboard can refresh metrics without heavy client-side processing.

MQTT is used for asynchronous scooter communication instead of forcing all device operations through synchronous HTTP calls. This helps reduce pressure on the main request path and better matches the device-control scenario.

7.3 Maintainability design

The multi-module backend structure separates common utilities, data objects, and runnable business services. The frontends are also separated by role, so the rider app, dispatcher

app, and admin dashboard can evolve independently. The use of MyBatis mappers and explicit DTO/VO classes makes the code easier to trace from request to persistence.

7.4 Reliability design

The system uses unified result wrapping and global exception handling, which makes failures easier to interpret on the client side. The ride flow is stateful and guarded by business checks such as one active ride per user and unpaid-order blocking. MQTT retry and online-status support further improve operational reliability for scooter commands.

7.5 Known limitations

The current implementation is still a course project, so some aspects are simplified:

- scooter hardware communication is simulated rather than deployed on full-scale devices;
- map display and zone calculations rely on the configured map service and current data format;
- some admin analytics are produced through direct queries rather than a dedicated reporting subsystem;
- the UI design is functional but not yet standardized as a complete design system.

Aspect	Design choice	Effect
Security	JWT interceptors and role-specific paths	Reduces unauthorized access across user roles.
Performance	Indexed relational tables and map-scale filtering	Keeps requests bounded and predictable.
Maintainability	Multi-module backend and DTO/VO structure	Makes the codebase easier to trace and extend.
Reliability	Global exception handling and MQTT retry support	Improves failure visibility and command delivery.

Table 21: Summary of non-functional design decisions.

8 Summary

Panda Scooter demonstrates a complete end-to-end shared scooter workflow that covers rider operations, dispatcher operations, and administrative management. The system combines a layered architecture, role-separated frontends, a shared Spring Boot backend, and a relational database schema that maps directly to the major business entities in the platform.

From a design perspective, the project shows that the three user roles can be supported in a consistent and maintainable way. The modular backend structure, the unified API

response pattern, and the clear separation between business services and integration services all help reduce coupling and make the system easier to extend.

The current implementation is already able to support the main business flow of a scooter-sharing platform, but there is still room for improvement in areas such as analytics, device integration, automated testing, UI consistency, and scalability. These directions define the next stage of development if the project is continued beyond the current course scope.